2 When you delete an object with a `FileField` or `ImageField`, Django does not remove the associated file. Write a `cleanup` command that searches through all the `Musician` and `Venue` objects for their associated pictures and compares this to the uploaded files, removing any orphans. Hint 1: use a Python `set` to contain all picture references and another to contain all file references, and then use the `set.difference()` method to determine the orphan list. Hint 2: convert the picture values into `pathlib.Path` objects before comparing them. Hint 3: test your code a lot, printing out results instead of deleting files until you're absolutely sure it works.

## Summary

- A management command is a Python script that allows you to interact with your Django project from the command-line.
- You've used Django management commands throughout the book already, to create a superuser, create and migrate ORM tables, launch a Django-aware REPL, launch your database's command-line tool, save and load fixtures, and run your unit tests.
- Django comes with many management commands that give you the ability to create superusers, change passwords, import and export data, manage database interactions, run tests, and more.
- The `changepassword` management command allows you to change a user's password. It does not require authentication to do so, as Django expects people with access to the command-line to be administrators.
- The `diffsettings` management command displays the difference between the default configuration of a project and your own.
- The `flush` management command wipes all data from your database but keeps the table structure and current migration state. It is useful when developing a new project to clear out test data.
- The `sqlflush` and `sqlmigrate` management commands print out the SQL that would be run for the corresponding `flush` and `migrate` commands.
- Each management command is its own file associated with a Django app, living inside the app's management/commands directory. The name of the file is the name of the command. The directories are Python modules and require a __init__.py file.
- Django expects each management command file to contain a class that extends from `BaseCommand`.
- The `.handle()` method in your management command class is the entry point for performing an action. It is invoked by Django when the management command gets run.
- Instead of using `print()` in a management command, it is recommended to use `self.stdout.write()` and `self.stderr.write()`. Using these proxies to the output streams makes it easier to write tests for your management command.